Tema 0

Introducción a la programación de dispositivos móviles

Plataformas

Android

iOS

Otros

Huawei

iOS

1984

Presentación del Macintosh

El 24 de Enero de 1984 Steve Jobs presenta el primer ordenador con interfaz gráfica y sistema operativo MacOS

2007

iPhone Original

Sale a la venta el iPhone original con sistema operativo iPhone OS y que combinaba 3 productos: teléfono, iPod touch y cámara

2010

Del iPhone OS a IOS

Cambio de nombre del sistema operativo de “iPhone OS” a iOS

Android

2005

Google compra Android Inc.

En julio de 2005, la multinacional Google compra Android Inc.

2007

Versión 1.0 Apple Pie

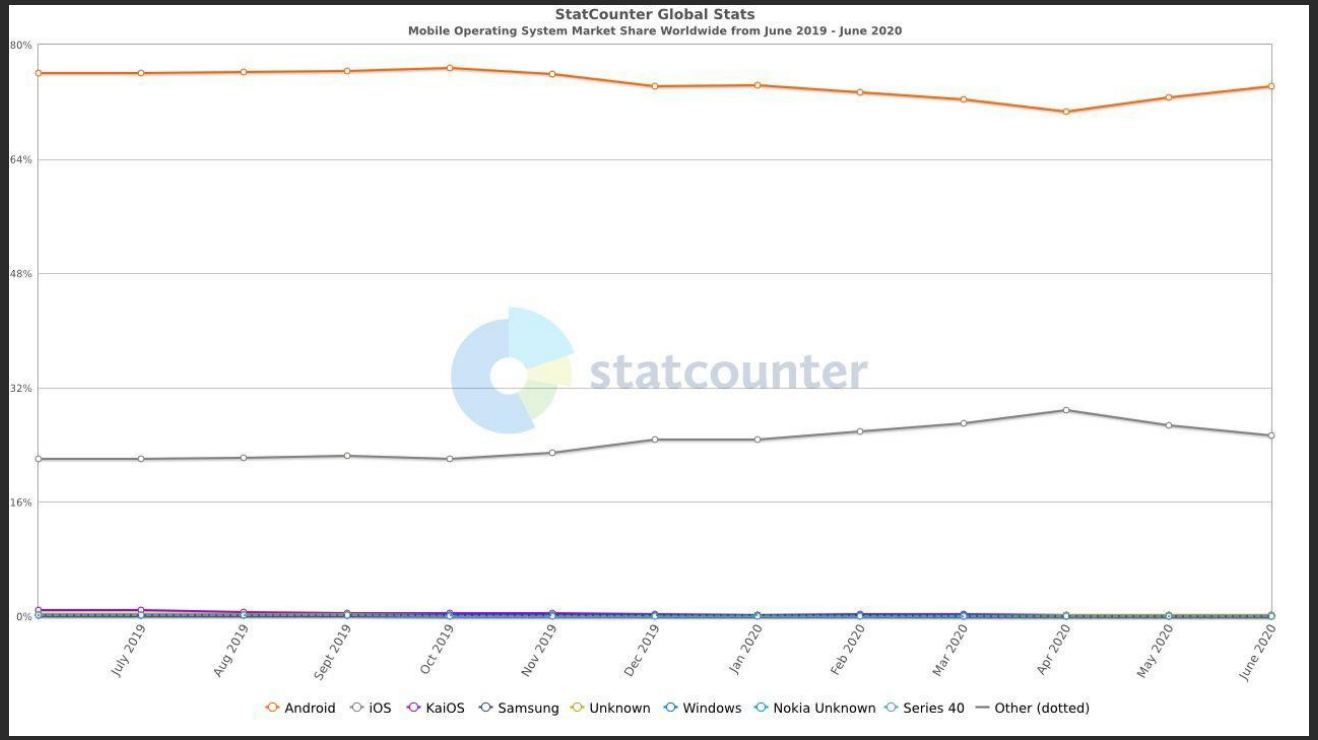
En 2007 se lanza la primera versión del sistema operativo de Android: Apple Pie

2008

Primer móvil Android

Los primeros terminales con Android se venden en el año 2008

Android vs iOS

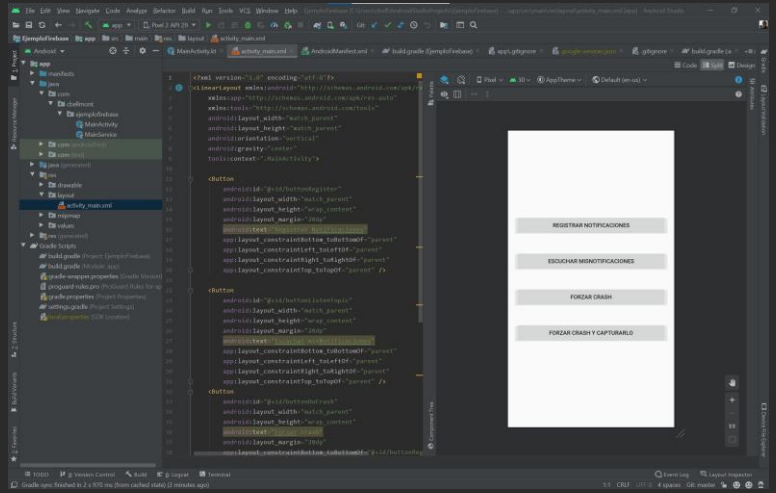


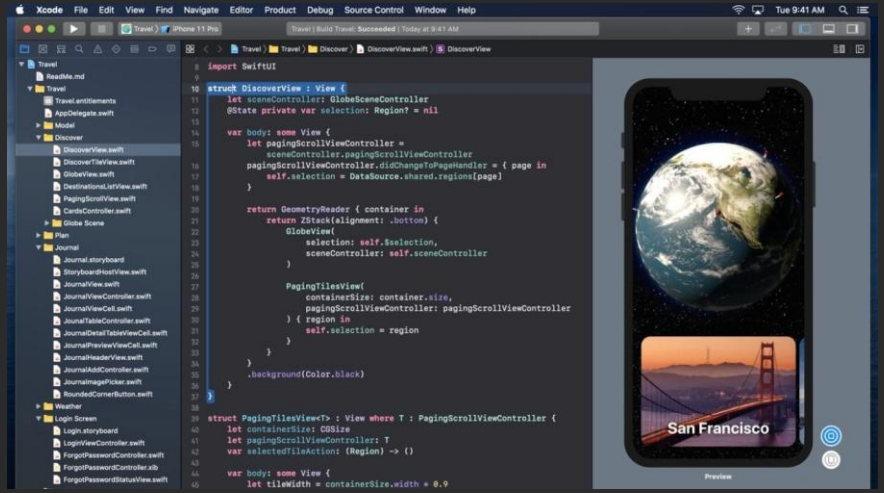
Tipos de Aplicaciones

- Aplicaciones nativas
  - Código nativo
- WebApp
  - Código HTML + JavaScript
- Híbridas (WebApp Nativa)
  - PhoneGap
  - Cordova
  - **Flutter**

Tipos de Aplicaciones: Aplicaciones Nativas I

Son aplicaciones desarrolladas íntegramente para un sistema operativo, denominados SDK (software development kit), para dispositivos móviles.





Tipos de Aplicaciones: Aplicaciones Nativas II

Ventajas:

- Acceso a las funcionalidades del dispositivo.
- Mejor experiencia de usuario.
- Visibilidad en las tiendas de aplicaciones.
- Envío de notificaciones al usuario.
- Alto nivel de actualización en la aplicación.

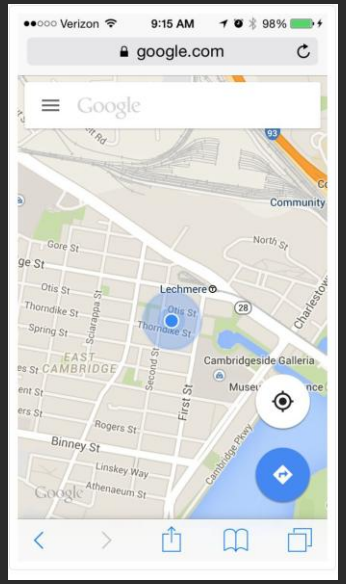
Tipos de Aplicaciones: Aplicaciones Nativas III

Inconvenientes:

- Diferentes habilidades/idiomas/herramientas para cada plataforma de destino.
- Tienden a ser más caras de desarrollar (Al menos sobre le papel).
- El código del cliente no suele ser reutilizable en otras plataformas.

Tipos de Aplicaciones: WebApp I

Son aplicaciones desarrolladas con tecnología web (html,css, js) y adaptadas en su diseño y funcionalidad a su ejecución en dispositivos móviles.



Tipos de Aplicaciones: WebApp II

Ventajas:

- Reutilización del código.
- Proceso de desarrollo más sencillo.
- No necesitas aprobación externa.
- El usuario siempre dispone de la última versión.
- Reutilización de sitios ya diseñados.

Tipos de Aplicaciones: WebApp III

Inconvenientes:

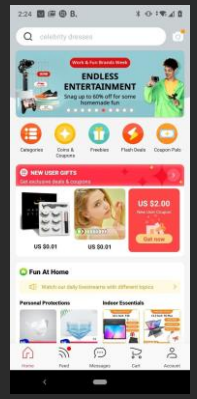
- Requiere conexión.
- Acceso limitado a las funcionalidades.
- Experiencia del usuario peor por las constantes conexiones al servidor.
- Peor visibilidad en las ventas.

Tipos de Aplicaciones: Híbridas I

WebApp Nativa (Híbrida): Es una combinación de las dos anteriores. Desarrolladas con lenguajes de la web, que permite su ejecución en diferentes plataformas, pueden acceder a gran parte de las características del hardware del dispositivo. Se puede distribuir en las tiendas y para ello se utiliza un framework para desarrollo multiplataforma.

Ejemplo de Frameworks:

PhoneGap y Cordova.



Tipos de Aplicaciones: Híbridas II

Ventajas:

- Distribución en tienda.
- Instalación nativa pero construida con JS, HTML y CSS.
- Mismo código para múltiples plataformas.
- Acceso a parte del hardware.

Tipos de Aplicaciones: Híbridas III

Desventajas:

- Experiencia del usuario más web que de una aplicación nativa.
- Diseño visual (normalmente) no adaptado al S.O

Tecnologías para desarrollo

Vamos a analizar las siguientes:

- Xamarin
- PhoneGap
- Flutter
- Ionic
- Android Studio

Diferencias entre programar en Windows y Android

1. Limitaciones del hardware.

2. Limitaciones de la plataforma.

3. Expectativas del usuario.

1- Limitaciones del hardware

Diversidad de modelos (Atención a la Gama baja):

- Pantallas más pequeñas
- Procesadores menos potentes
- Almacenamiento
- Memoria RAM más limitada

Redes móviles:

- Tasa de transferencia de datos menor.
- Conexiones de datos menos estables.
- Coste de transferencia de datos.

2- Limitaciones del software

Arquitectura del SO - Permisos, ciclo de vida, etc.

Nuevos entornos de desarrollo -

Curva de aprendizaje elevada - Uso de componentes más avanzados.

Fragmentación - Android.

Personalización de los fabricantes - Android.

3- Expectativas del usuario

Interfaz de usuario adaptada a la plataforma.

Fluidez.

Sencillez.

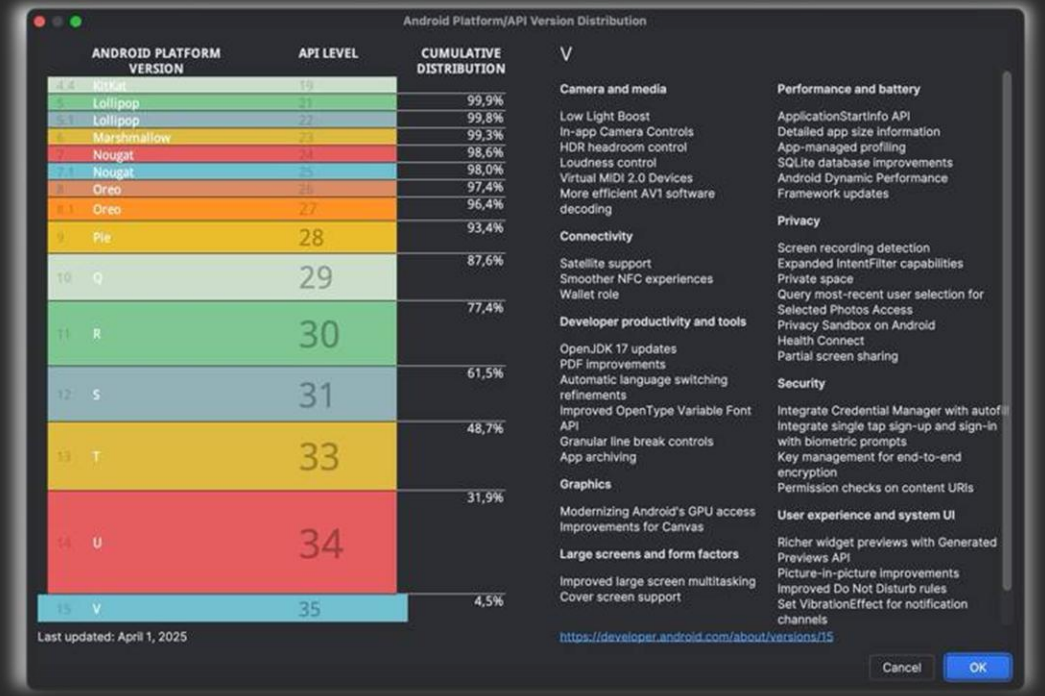
“Un usuario abre el 90% de aplicaciones que se descarga una única vez”

“Más del 50% de los usuarios cerrará la aplicación si no se ha abierto en 3 segundos”

Google

Versiones Android

Versiones en Android y lo que aporta cada una de ellas.



Lenguajes de Programación en iOS

Objective-C

Creado en **1980** y adoptado en **1988** por NextStep, en **1996** Apple compra NextStep incorporandolo adoptando Objective-C como lenguaje principal

Ejemplo código:

```
if ([delegate respondsToSelector:
@selector(application:willFinishLaunchingWi
thOptions:)]) {
     [delegate application:app
willFinishLaunchingWithOptions:options];
}
```

Swift

Creado por Apple en **2014** para mejorar el desarrollo de aplicaciones tanto para Mac como iOS.

Ejemplo código:

```
delegate.application?(app,
willFinishLaunchingWithOptions: options)
```

Lenguajes de Programación

Java

Lenguaje de programación orientado a objetos nacido en **1995** de la mano de James Gosling de **Sun Microsystems.**

Su ejecución requiere de una máquina virtual o navegador web.

Ha sido un lenguaje oficial de Android desde sus comienzos.

Kotlin

Lenguaje de programación orientado a objetos nacido en **2012** de la mano de **JetBrains**.

En 2017 Google lo elige como **lenguaje oficial para Android** junto a **Java**.

Lenguajes de Programación

Java

Ejemplo código:

```
View rootView =
inflater.inflate(R.layout.fragment_data,
                                container, false);
View btnShowName =
rootView.findViewById(R.id.tvTextDetail);
((Button) btnShowName).setOnClickListener( view
-> {
        if (mCallback != null) {
                mCallback.onFragmentInteraction("H
        OLA!");
        }
});
```

Kotlin

Ejemplo código:

```
findViewById<Button>(R.id.btnShowName).setOn
ClickListener {
        mCallback?.onFragmentInteraction("HOLA!")
}
```

Lenguajes de Programación

C y C++

Es posible recurrir a C y C++ para la realización de ciertas tareas que requieran un nivel de optimización extremadamente elevado.

Su uso no está muy generalizado y no se recomienda.

En todo caso, **Android ofrece el NDK (Native Developer Kit)** para desarrollar en C.

Lenguajes de Programación

Si ya conozco Java… ¡¿Por qué Kotlin?!

Kotlin es un lenguaje más moderno que Java pensado a las necesidades actuales:

- Más eficiente de escribir - Hacer más cosas escribiendo menos.
- Previene de forma más eficiente los fallos en tu código, en especial los “null”.
- Es la recomendación de Google, la mayor parte de la documentación y tutoriales modernos usan Kotlin cómo lenguaje principal.
- Todas las nuevas funciones de Android están pensadas para una fácil integración en Kotlin.

Lenguajes de Programación

Artículos

- Leed los siguientes artículos.

  Así se convirtió kotlin en lenguaje de referencia para desarrolladores Android

  Swift vs Objective-C para el desarrollo de aplicaciones iOS

Preguntas

Busca información sobre la coexistencia de Objective-C y Swift y de Java y Kotlin en un mismo proyecto.

- ¿Pueden coexistir?
- ¿De manera sencilla?
- ¿Es posible transformar código Java en Kotlin?